STIL- OCH TEKNIKGUIDE: STILISERAD VEGETATION

Estetiska Kategorier, Öppen Pipeline & Algoritmisk Generatorarkitektur

Pipelines: Blender · ComfyUI · Material Maker

Target Engine: Unreal Engine / Godot / Unity

Metod: Open-Source Workflows

1. De Fyra Huvudkategorierna inom Stiliserad Vegetation

Stiliserad vegetation handlar om medveten abstraktion av form, ljus och färg. Valet av estetik avgör geometriuppbyggnad, texturkrav och motorns shader-komplexitet.

1. Painterly / Ghibli-stil

Inspirerad av akvarell och bakgrundskonst. Bygger på mjuka, stora "lövmoln" (clumps) med lågfrekvent brus. Undviker enskilda blad till förmån för volymskuggning via sfäriska normaler.

2. Clean Anime / Genshin-stil

Grafisk och ren estetik med spetsiga lövkort. Använder *Runtime Virtual Textures (RVT)* för att sömlöst färgmatcha träd- och gräsrötter mot markens exakta nyans.

3. Hand-Painted (Klassisk WoW)

Traditionell teknik där all belysning, skuggor (AO) och highlights är inmålade direkt i basfärgen. Körs unlit utan beräkning av normaler, roughness eller dynamiskt ljus.

4. Stiliserad PBR (Zelda BotW / Kena)

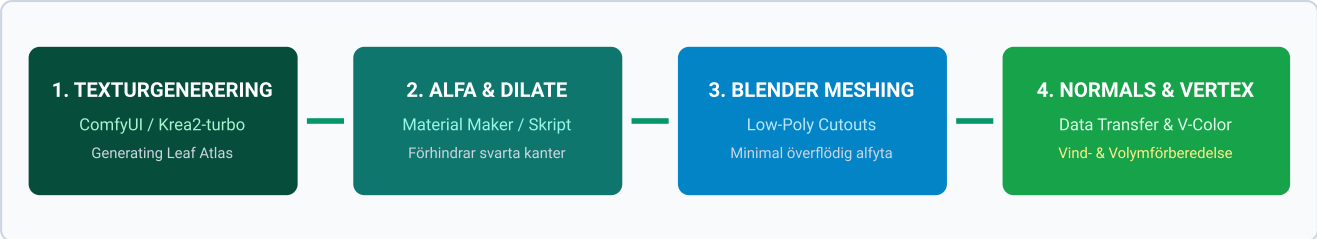
Hybrid där geometrin är förenklad men ytor reagerar fysikaliskt korrekt på ljus. Utnyttjar *Subsurface Scattering (SSS)* för att låta solljus lysa igenom bladverket.

2. Jämförande Kravmatris för Renderings-Pipelinen

Kategori	Geometri & Polygonsida	Shader & Belysningsmodell	Texturkrav	Normal-hantering
Painterly (Ghibli)	Mellan (Lövkluster-volymer)	Soft Volumetric / Smooth Toon	Lågfrekvent Noise + Gradient	Sfärisk Överföring (Proxy Mesh)
Clean Anime	Låg (Minimal överflödig alfyta)	Toon Shader + RVT Blend	Spetsiga Alfa-kort + Atlas	Justerade uppåt / Sfäriska
Hand-Painted	Mycket låg (Enkla kort & stam)	Unlit / Diffuse-Only	Högkontrast Handmålad Albedo	Inga normalmaps krävs
Stiliserad PBR	Mellan till Hög (Detaljerade kort)	PBR med Subsurface Scattering	Albedo, Roughness, Normal Map	Unika kortnormaler + Volym

3. Öppen Produktions-Pipeline & Atlas-preparering

Pipelinen bygger helt på fri mjukvara. För att undvika fula ränder kring genomskinliga kanter vid mipmapping krävs en strikt kantexpansionssteg (Dilate) innan montering i Blender.



4. Volymskuggning & Vinddata-arkitektur

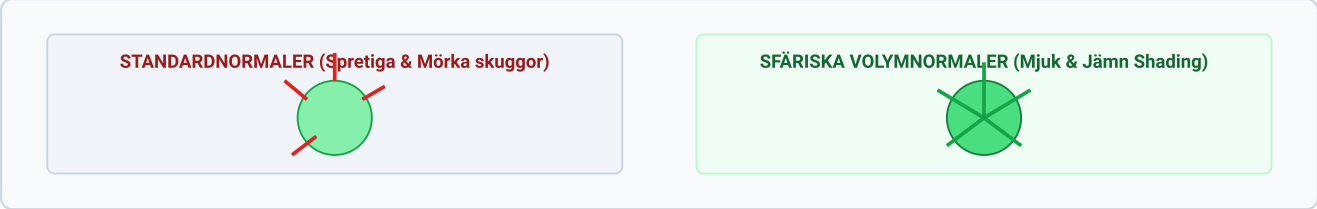
Sfärisk Normal-överskrivning

Standardnormaler på vinklade lövkort ger ett fläckigt och platt intryck med svarta skuggor. Genom Blenders **Data Transfer Modifier** överförs normaler från en omslutande proxy-sfär. Hela trädkronan skuggas därefter som en mjuk, sammanhängande volym.

Vertex Color Wind Masking

Spelmotorers vind-shaders kräver styrdata i modellens vertex-färger:

- **Röd (R):** Höjd över marken (storskalig svajning).
- **Grön (G):** Avstånd från lokal gren-pivot (lövfladder).
- **Blå (B):** Slumpmässigt fas-ID per grenkluster.



5. Algoritmisk Generatorarkitektur för Träd

En kodbaserad växtgenerator bygger på *Space Colonization* för grenstrukturen kombinerat med automatisk beräkning av tjocklek enligt Da Vincis grenregel samt vektoriell normal-projektion.

Matematisk Formel för Sfärisk Normal-överskrivning:

$$\hat{N}_{final} = \text{normalize}((1 - \alpha) \cdot \hat{N}_{orig} + \alpha \cdot \text{normalize}(P_v - C))$$

där C är kronans/klustrets centrumkoordinat, P_v är vertexens position och $\alpha \in [0.75, 0.90]$ är viktfaktorn för volymutjämnandet. Grenradie beräknas via: $r_{parent}^{\gamma} = \sum r_{child}^{\gamma}$ (där $\gamma \approx 2.0$).

6. Python/Blender Generator-implementering

```
# Procedurell beräkning av vertex-färger, sfäriska normaler & atlas-UV import bpy, mathuti
ls def process_procedural_foliage(mesh_obj, canopy_center, max_height): mesh = mesh_obj.data #
1. Skapa eller hämta Vertex Color Layer vcol = mesh.vertex_colors.get("Col") or mesh.vertex_colo
rs.new(name="Col") # 2. Iterera över alla loopar och beräkna vind- & normal-data for poly in mes
h.polygons: for loop_idx in poly.loop_indices: v_idx = mesh.loops[loop_idx].vertex_index pos = m
esh.vertices[v_idx].co # R-kanal: Höjdfaktor [0.0 -> 1.0] r_wind = min(max(pos.z / max_height,
0.0), 1.0) # G-kanal: Avstånd från kronas centrum (lövflex) dist_center = (pos - canopy_center).
length g_flex = min(dist_center / 3.5, 1.0) # Sätt Vertex Color (R, G, B, Alpha) vcol.data[loop_
idx].color = (r_wind, g_flex, 0.5, 1.0) # 3. Sfärisk Normal-överskrivning alpha = 0.85 custom_no
rmals = [] for v in mesh.vertices: orig_norm = v.normal sphere_norm = (v.co - canopy_center).nor
malized() final_norm = ((1.0 - alpha) * orig_norm + alpha * sphere_norm).normalized() custom_nor
mals.append(final_norm) mesh.normals_split_custom_set_from_vertices(custom_normals) mesh.use_aut
o_smooth = True
```

UV Atlas-packningsregel: Dela in atlasen i 2×2 eller 4×4 delregioner. Vid procedurell generering slumpas en region per lövkort genom att addera en offset $(U_{\text{offset}}, V_{\text{offset}})$ till kortets lokala UV-koordinater.